

2021-2022 Resit Exam

Answer Key

CPT104 Operating Systems

Answer Key with English & Chinese Explanations

Question 1: Fundamentals (45 marks)

(a) [6 marks]

English Answer

Page: A fixed-size block of logical (virtual) address space. Pages exist in the process's virtual memory.

Frame: A fixed-size block of physical memory (RAM). Frames are actual physical memory locations.

Key differences:

- Pages and frames are the same size but exist in different address spaces.
- The OS maps pages to frames via the page table.
- Virtual memory allows total pages to exceed total frames (disk backing store).
- Pages are a logical concept; frames are physical reality.

中文解释

页 (Page) : 逻辑/虚拟地址空间的固定大小块, 存在于进程的虚拟内存中。

帧 (Frame) : 物理内存 (RAM) 的固定大小块, 是实际的物理内存位置。

大小相同但存在于不同空间, 通过页表映射。虚拟内存允许总页数超过总帧数。

(b) [6 marks]

English Answer

Context switch overhead in real-time systems is significant for three reasons:

- (1) Wasted CPU time: Every context switch consumes CPU cycles that could be used for task execution.
- (2) Accumulated overhead: In periodic real-time tasks with frequent switches, the overhead accumulates and reduces the effective utilization budget available for actual work.
- (3) Unpredictability: Variable context switch times make it harder to provide guaranteed deadline satisfaction. Real-time scheduling algorithms typically assume negligible context switch overhead.

中文解释

- (1) 浪费CPU时间：每次上下文切换都消耗可用于任务执行的CPU周期。
- (2) 开销累积：高频周期任务的频繁切换累积的开销减少了实际可用的CPU利用率预算。
- (3) 不可预测性：可变的切换时间使得保证截止时间更加困难。

(c) [12 marks]

English Answer

Three methods to ensure system security through backups and patches:

- (1) Automated Backups:
 - Schedule daily/weekly backups using tools like Time Machine, OneDrive, or cloud services.
 - Follow the 3-2-1 rule: 3 copies, 2 different media types, 1 offsite.
 - Test backup restoration regularly.
- (2) Patch Management:
 - Enable automatic updates for OS and applications.
 - Unpatched vulnerabilities are the primary attack vector for most breaches.
 - Maintain a patching schedule and audit compliance.
- (3) Data Encryption:
 - Encrypt sensitive files individually.
 - Use full-disk encryption as baseline protection.
 - Encryption serves as the last line of defense if backups are compromised.

中文解释

- (1) 自动备份：使用Time Machine/OneDrive/云服务定期备份。遵循3-2-1规则（3份副本、2种介质、1份异地）。定期测试恢复流程。
- (2) 补丁管理：启用操作系统和应用程序的自动更新。强调未修补漏洞是大多数安全事件的主要攻击入口。维护补丁计划并审计合规性。
- (3) 数据加密：单独加密敏感文件。使用全盘加密作为基线防护。加密是备份被攻破时的最后防线。

(d) [4 marks]

English Answer

x starts at 0. Process P1 executes $x = x + 1$ three times. Process P2 executes $x = x + 1$ three times.

Without synchronization, $x = x + 1$ is NOT atomic (it involves read, increment, write). Concurrent execution can cause

lost updates.

Possible final values of x: 2, 3, 4, 5, 6

- Maximum = 6 (fully sequential, no overlap)
- Minimum = 2 (maximal interleaving: both processes read 0, both write 1, then both read 1, both write 2)

中文解释

x初始为0, P1和P2各执行3次 $x=x+1$ 。

没有同步时, $x=x+1$ 不是原子操作(读-加-写), 并发执行可能导致更新丢失。

x可能的最终值: 2, 3, 4, 5, 6

最大值=6(完全顺序执行), 最小值=2(最大重叠交错)。

(e) [12 marks]

English Answer

Virtual Machines - Two arguments FOR:

- (1) Isolation and Security: A crash or compromise in one VM does not affect other VMs or the host. Each VM is sandboxed.
- (2) Multiple OS Environments: A single physical machine can run multiple different operating systems simultaneously (e.g., Windows, Linux, macOS on the same hardware).

Two arguments AGAINST:

- (1) Performance Overhead: The hypervisor abstraction layer adds CPU, memory, and I/O overhead. Bare-metal performance is not achievable.
- (2) Complexity and Cost: Managing multiple VMs increases administrative complexity. Licensing costs multiply (each VM may need its own OS license).

中文解释

支持虚拟机的两个论点:

- (1) 隔离性和安全性: 一个VM的崩溃或被攻破不会影响其他VM或宿主机, 每个VM都是沙箱化的。
- (2) 多操作系统环境: 单台物理机可同时运行多个不同操作系统。

反对虚拟机的两个论点:

- (1) 性能开销: Hypervisor抽象层增加CPU、内存和I/O开销, 无法达到裸机性能。
- (2) 复杂性和成本: 管理多个VM增加管理复杂度, 许可成本成倍增加。

(f) [5 marks]

English Answer

Thread switching is cheaper than process switching because:

- (1) Shared address space: No need to flush the MMU (Memory Management Unit) or TLB (Translation Lookaside Buffer) during a thread switch.
- (2) Shared resources: Threads share open files, signal handlers, and other OS resources, so fewer kernel data structures need to be updated.
- (3) Smaller context: Thread switching only saves the register set and stack pointer, compared to process switching which must save the entire PCB (Process Control Block), memory maps, and file descriptors.

中文解释

线程切换比进程切换更便宜，原因：

- (1) 共享地址空间：线程切换无需刷新MMU和TLB。
- (2) 共享资源：线程共享打开的文件、信号处理等OS资源，需要更新的内核数据结构更少。
- (3) 上下文更小：线程切换只保存寄存器组和栈指针，进程切换需保存整个PCB、内存映射和文件描述符。

Question 2: CPU / Memory / Disk Scheduling (31 marks)

(a) [6 marks]

English Answer

Reference string: 0 1 7 0 1 2 0 1 2 3 2 7 1 0 3 1 0 3, Frames = 3

FIFO trace (replace oldest page):

```

Ref=0: [0]      F
Ref=1: [0,1]    F
Ref=7: [0,1,7]  F
Ref=0: [0,1,7]  H
Ref=1: [0,1,7]  H
Ref=2: [2,1,7]  F (replace 0)
Ref=0: [2,0,7]  F (replace 1)
Ref=1: [2,0,1]  F (replace 7)
Ref=2: [2,0,1]  H
Ref=3: [3,0,1]  F (replace 2)
Ref=2: [3,2,1]  F (replace 0)
Ref=7: [3,2,7]  F (replace 1)
Ref=1: [3,2,7]  H
Ref=0: [0,2,7]  F (replace 3)
Ref=3: [0,3,7]  F (replace 2)
Ref=1: [1,3,7]  F (replace 0)
Ref=0: [1,0,7]  F (replace 3)
Ref=3: [1,0,3]  F (replace 7)
  
```

Total page faults = 15

中文解释

使用FIFO算法，3个帧。FIFO替换最早进入的页面。

逐步追踪如上。

共15次缺页。

(b) [12 marks]

English Answer

This question requires a memory allocation diagram showing the current state of memory partitions. Without the visual diagram from the original exam paper, a detailed answer cannot be provided.

General approach for memory allocation problems:

- Identify free and allocated partitions
- Apply First-Fit, Best-Fit, or Worst-Fit as specified
- Show the resulting memory layout after each allocation/deallocation
- Calculate remaining free space and any fragmentation

中文解释

此题需要内存分配图来展示分区状态。无法从文本提取中获取具体的内存布局数据。

内存分配问题的一般解题方法：

- 识别空闲和已分配分区
- 按要求应用首次适应/最佳适应/最差适应算法
- 展示每次分配/释放后的内存布局
- 计算剩余空间和碎片情况

(c) [7 marks]

English Answer

Processes: P1(arrival=0, burst=3, priority=3), P2(arrival=2, burst=7, priority=4), P3(arrival=3, burst=5, priority=1), P4(arrival=5, burst=9, priority=2)

Lower number = higher priority. Non-preemptive priority scheduling.

Gantt chart:

| P1(0-3) | P3(3-8) | P4(8-17) | P2(17-24) |

At time 0: Only P1 is available -> P1 runs (0-3)

At time 3: P2(pri=4), P3(pri=1) available -> P3 runs (3-8)

At time 8: P2(pri=4), P4(pri=2) available -> P4 runs (8-17)

At time 17: P2 runs (17-24)

Waiting times: P1=0, P3=5, P4=12, P2=15

Average waiting time = $(0+15+5+12)/4 = 8.0$

中文解释

非抢占式优先级调度（数字越小优先级越高）。

甘特图：P1(0-3) → P3(3-8) → P4(8-17) → P2(17-24)

时间0：只有P1就绪，P1运行。

时间3：P2(优先级4)和P3(优先级1)就绪，P3优先级高，P3运行。

时间8：P2(4)和P4(2)就绪，P4运行。

时间17：P2运行。

等待时间：P1=0, P3=5, P4=12, P2=15

平均等待时间 = $(0+15+5+12)/4 = 8.0$

(d) [6 marks]

English Answer

Requests: 17, 30, 24, 37, 15, 27, 11, 75, 20, 5

Disk size = 80 cylinders, Initial head = 28

SSTF (Shortest Seek Time First) - always move to the nearest request:

28 -> 27 (distance=1)
27 -> 24 (distance=3)
24 -> 20 (distance=4)
20 -> 17 (distance=3)
17 -> 15 (distance=2)
15 -> 11 (distance=4)
11 -> 5 (distance=6)
5 -> 30 (distance=25)
30 -> 37 (distance=7)
37 -> 75 (distance=38)

Total movement = $1+3+4+3+2+4+6+25+7+38 = 93$ cylinders

中文解释

SSTF (最短寻道时间优先) 算法, 每次移动到最近的请求。

28 → 27(1) → 24(3) → 20(4) → 17(3) → 15(2) → 11(4) → 5(6) → 30(25) → 37(7) → 75(38)

总移动量 = 93 柱面。

Question 3: Resource Allocation (12 marks)

English Answer

This question requires a Resource Allocation Graph (RAG) diagram.

Without the visual diagram from the original exam paper, a detailed analysis cannot be provided.

General approach for RAG questions:

- Identify process nodes (circles) and resource nodes (squares)
- Draw request edges (process -> resource) and assignment edges (resource -> process)
- Check for cycles: a cycle in a RAG with single-instance resources means deadlock
- If multiple instances of resources exist, a cycle is necessary but not sufficient for deadlock

中文解释

此题需要资源分配图 (RAG) 来分析。

无法从文本提取中获取图形数据, 无法提供详细答案。

RAG问题的一般解题方法:

- 识别进程节点 (圆圈) 和资源节点 (方块)
- 画出请求边 (进程→资源) 和分配边 (资源→进程)
- 检查是否存在环: 单实例资源的环意味着死锁
- 多实例资源时, 环是死锁的必要非充分条件

Question 4: OS in C (Producer-Consumer Problem) (12 marks)

(a) [2 marks]

English Answer

If the 'full' semaphore is removed from the producer-consumer solution:

The producer will never be blocked from writing, even when the buffer is full. It will overwrite existing data (buffer overflow). The consumer may read garbage/inconsistent data because the synchronization between producer and consumer is broken.

中文解释

移除full信号量后，生产者写入时不会被阻塞，即使缓冲区已满也会继续写入，导致缓冲区溢出。生产者和消费者之间的同步被破坏，消费者可能读取无效/不一致的数据。

(b) [5 marks]

English Answer

If 'empty' and 'mutex' are swapped in the producer code (producer acquires mutex before empty):

Result: DEADLOCK.

Scenario: Buffer is full (empty=0). Producer acquires mutex first, then tries to wait(empty). Since empty=0, the producer blocks while holding mutex. Consumer tries to consume and needs mutex, but mutex is held by the blocked producer. Circular wait condition is established.

This is a classic deadlock scenario in the producer-consumer problem.

中文解释

如果生产者代码中empty和mutex的顺序互换（先获取mutex再等待empty）：

结果：死锁。

场景：缓冲区满（empty=0）。生产者先获取mutex，然后等待empty。因为empty=0，生产者在持有mutex的情况下阻塞。消费者需要mutex才能消费，但mutex被阻塞的生产者持有。形成循环等待条件，经典死锁场景。

(c) [5 marks]

English Answer

If signal(full) and signal(empty) are swapped in the producer:

The synchronization logic becomes completely reversed. The producer incorrectly signals 'empty' (indicating a free slot when it actually just filled one) and the consumer incorrectly signals 'full' (indicating a filled slot when it actually just freed one).

This corrupts the buffer management: the semaphore values no longer accurately reflect buffer state, leading to potential buffer overflow, underflow, or deadlock depending on timing.

中文解释

如果生产者中signal(full)和signal(empty)互换：

同步逻辑完全反转。生产者错误地signal(empty) (刚填满一个槽位却说有空位)，消费者错误地signal(full) (刚释放一个槽位却说有数据)。

缓冲区管理将出错：信号量值不再准确反映缓冲区状态，可能导致溢出、下溢或死锁。